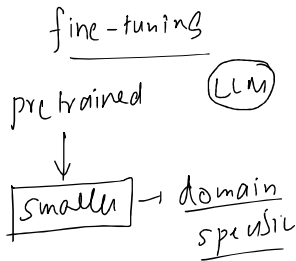
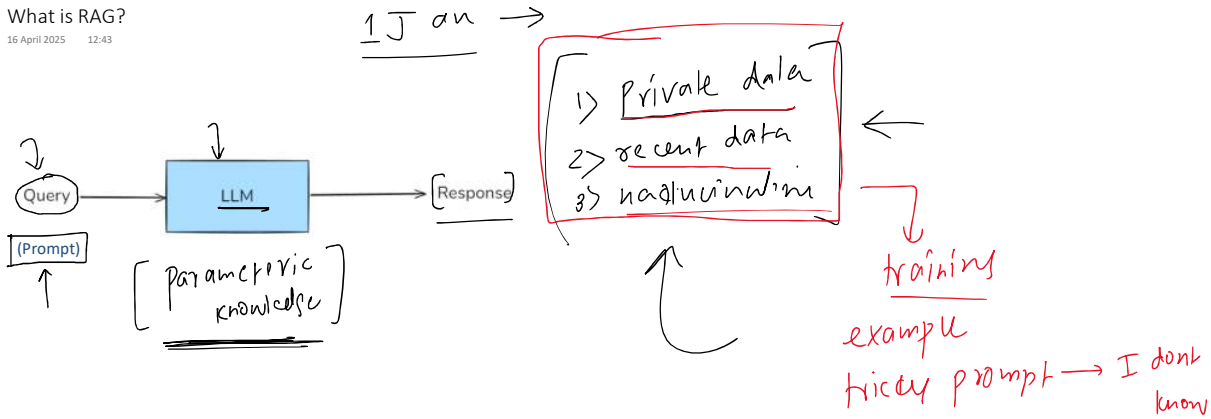


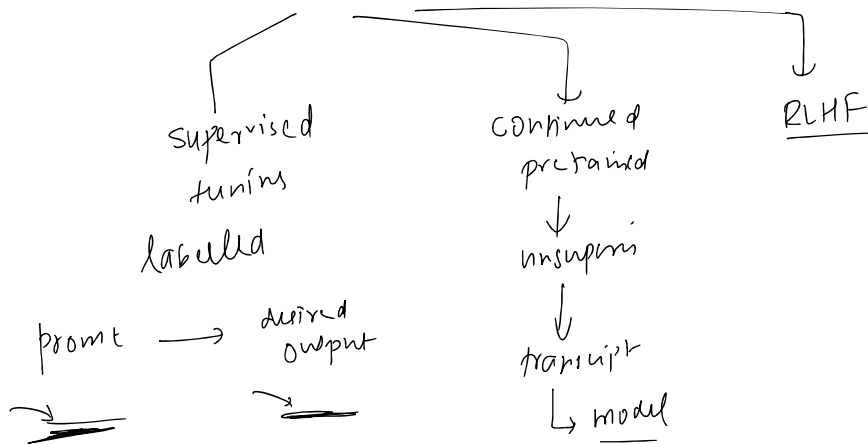
What is RAG?

16 April 2025 12:43



student - LLM
ends → pretrain
train → fine-tuning

1. Collect data	A few hundred - few hundred-thousand carefully curated examples (prompts → desired outputs).
2. Choose a method	Full-parameter FT, LoRA/QLoRA, or parameter-efficient adapters.
3. Train for a few epochs	You keep the base weights frozen or partially frozen and update only a small subset (LoRA) or all weights (full FT).
4. Evaluate & safety-test	Measure exact-match, factuality, and hallucination rate against held-out data; red-team for safety.



- 1) LLM → train
- 2) technical expertise
- 3)

In context learning

In-Context Learning is a core capability of Large Language Models (LLMs) like GPT-3/4, Claude, and Llama, where the model learns to solve a task purely by seeing examples in the prompt—without updating its weights.

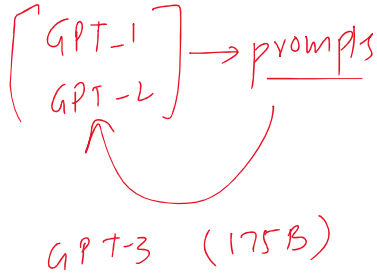
LLM → emergent property

Below are examples of texts labeled with their sentiment. Use these examples to determine the sentiment of the final text.

Text: I love this phone. It's so smooth. → Positive
Text: This app crashes a lot. → Negative
Text: The camera is amazing! → Positive

LLM → emergent prop

An **emergent property** is a behaviour or ability that suddenly appears in a system when it reaches a certain scale or complexity—even though it was not explicitly programmed or expected from the individual components.



Context

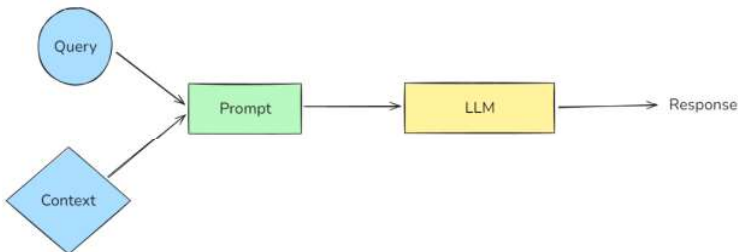
- Instead of just example tasks, retrieve background information, facts, documents, product manuals, etc.
- Inject that into the prompt to augment the model's knowledge

```

"""You are a helpful assistant.
Answer the question ONLY from the provided context.
If the context is insufficient, just say you don't know.

{context}
Question: {question}"""
  
```

RAG is a way to make a language model (like ChatGPT) smarter by giving it extra information at the time you ask your question.



Use these examples to determine the sentiment of the final text.

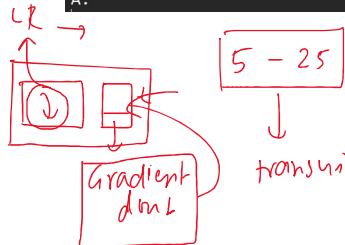
Text: I love this phone. It's so smooth. → Positive
 Text: This app crashes a lot. → Negative
 Text: The camera is amazing! → Positive
 Text: I hate the battery life. →

Label the named entities in the sentences (Person, Location, Organization):

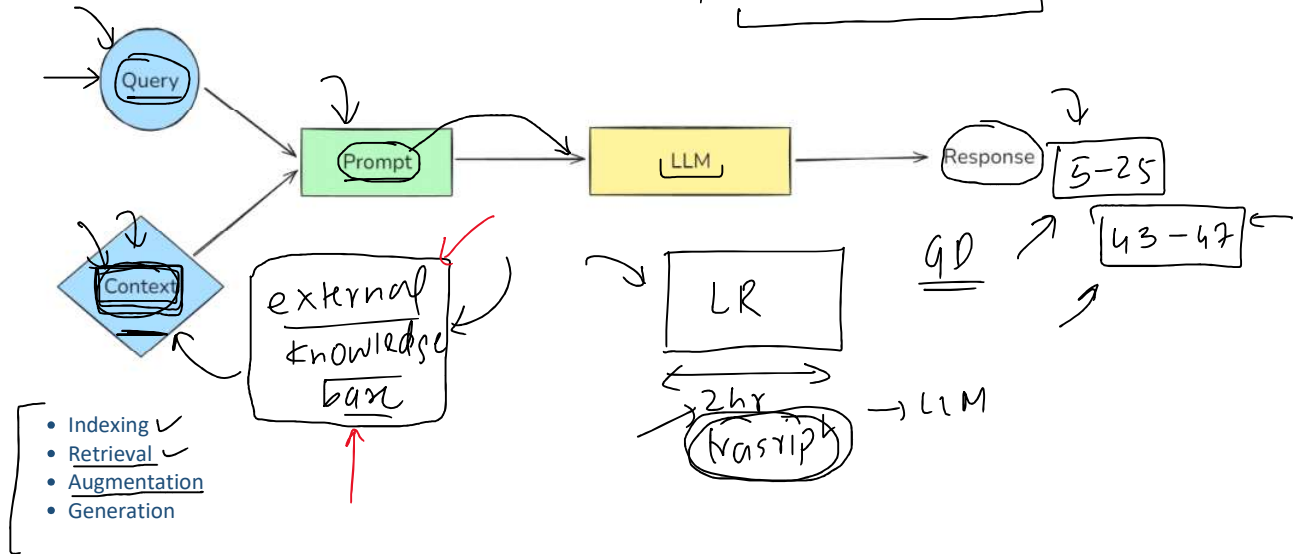
Sentence: ~~Elon Musk~~ founded ~~SpaceX~~.
 Entities: [Elon Musk → Person], [SpaceX → Organization]
 Sentence: The Eiffel Tower is in Paris.
 Entities: [Eiffel Tower → Location], [Paris → Location]
 Sentence: Sundar Pichai leads Google.
 Entities:

Solve the math problems step by step:

Q: John has 3 apples. He buys 4 more. How many apples does he have now?
 A: John had 3 apples. He bought 4 more. $3 + 4 = 7$. → 7
 Q: Sarah has 10 pens. She gives away 3. How many does she have left?
 A: Sarah had 10 pens. She gave away 3. $10 - 3 = 7$. → 7
 Q: A pizza is cut into 8 slices. Alex eats 3 slices. How many are left?
 A:



RAG → Information retrieval + text generation



Indexing - Indexing is the process of preparing your knowledge base so that it can be efficiently searched at query time. This steps consists of 4 sub-steps.

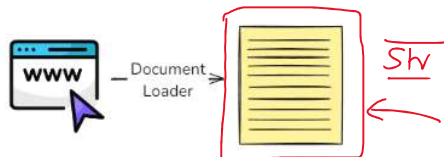
1. Document Ingestion - You load your source knowledge into memory.

Examples:

- PDF reports, Word documents
- YouTube transcripts, blog pages
- GitHub repos, internal wikis
- SQL records, scraped webpages

Tools:

- LangChain loaders (PyPDFLoader, YoutubeLoader, WebBaseLoader, GitLoader, etc.)



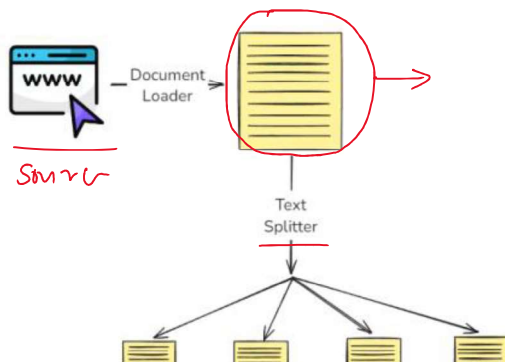
2. Text Chunking - Break large documents into small, semantically meaningful chunks

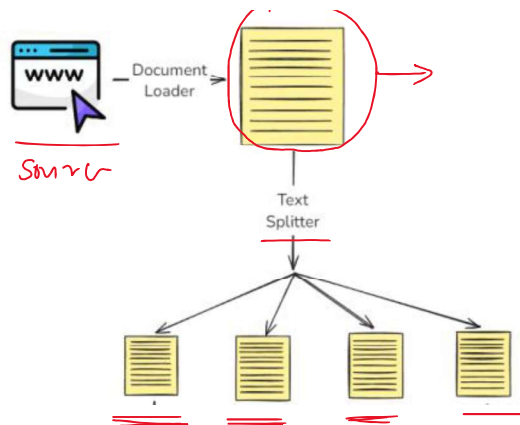
Why chunk?

- LLMs have context limits (e.g., 4K-32K tokens)
- Smaller chunks are more focused → better semantic search

Tools:

- RecursiveCharacterTextSplitter, MarkdownHeaderTextSplitter, SemanticChunker





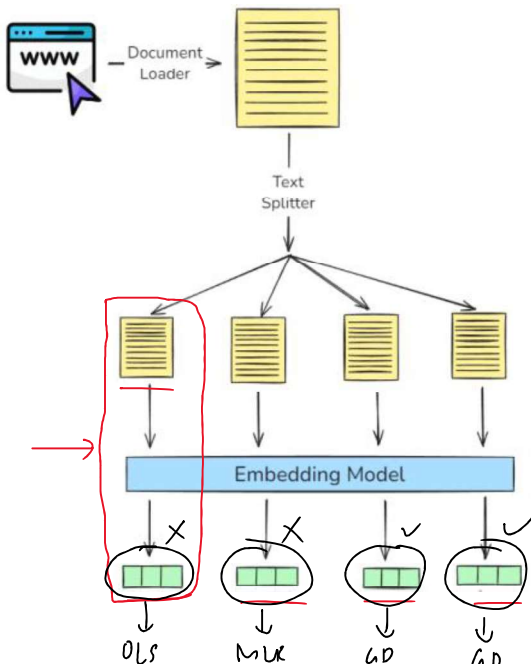
3. Embedding Generation - Convert each chunk into a dense vector (embedding) that captures its meaning.

Why embeddings?

- Similar ideas land close together in vector space
- Allows fast, fuzzy semantic search

Tools:

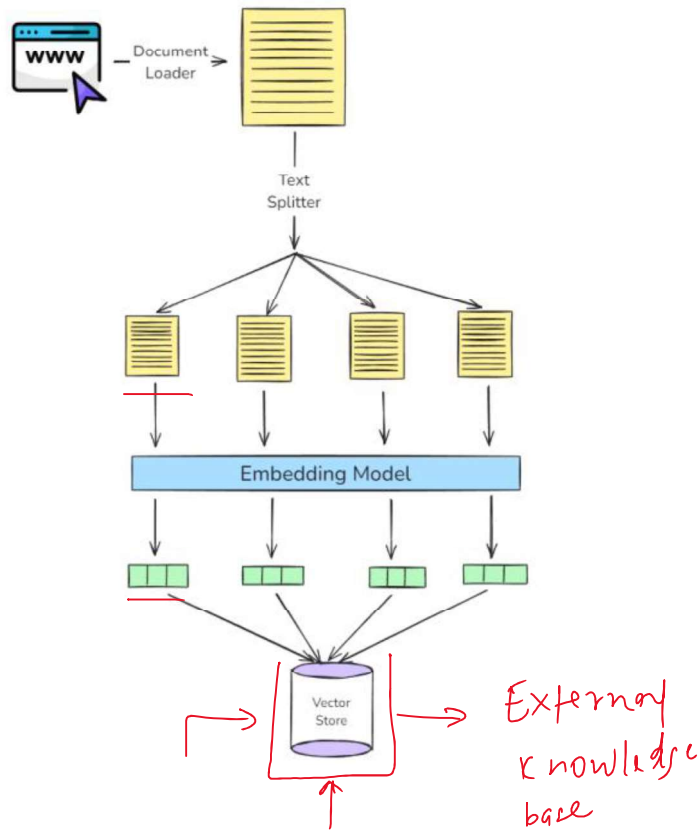
- OpenAIEmbeddings, SentenceTransformerEmbeddings, InstructorEmbeddings, etc.



4. Storage in a Vector Store - Store the vectors along with the original chunk text + metadata in a vector database.

Vector DB options:

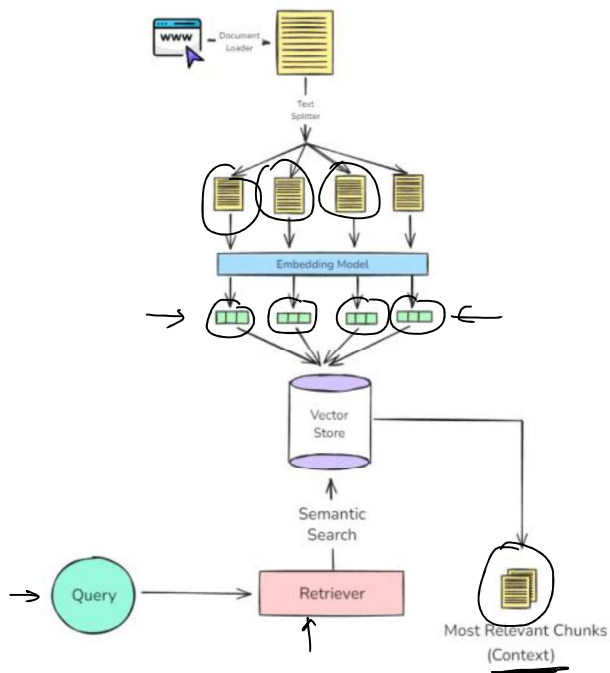
- Local: FAISS, Chroma
- Cloud: Pinecone, Weaviate, Milvus, Qdrant



Retrieval - Retrieval is the *real-time* process of **finding the most relevant pieces of information from a pre-built index** (created during indexing) based on the user's question.

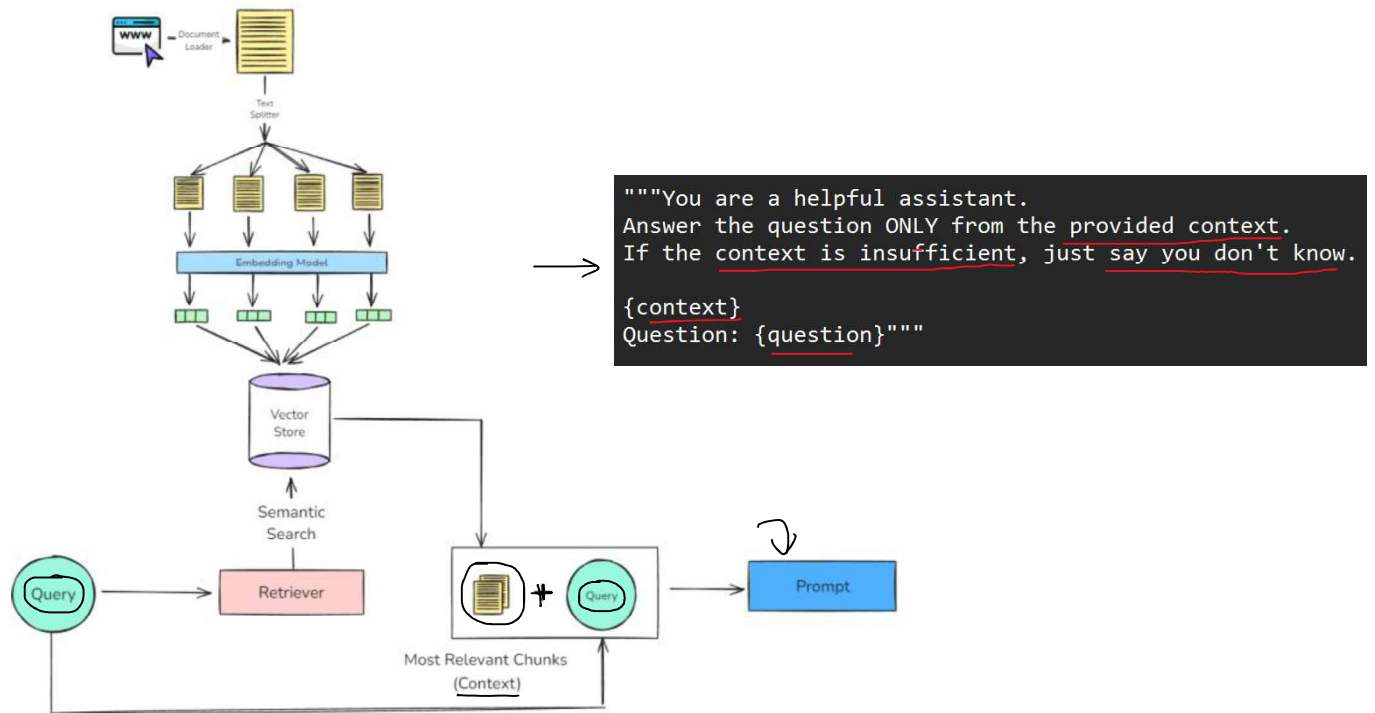
It's like asking:

"From all the knowledge I have, which 3-5 chunks are most helpful to answer this query?"



Step 1 → query → embeddings vector
 Step 2 → semantic search
 Step 3 → rankings
 Step 4 →

Augmentation - **Augmentation** refers to the step where the **retrieved documents** (chunks of relevant context) are **combined with the user's query** to form a new, enriched prompt for the LLM.



Generation - **Generation** is the final step where a **Large Language Model (LLM)** uses the **user's query** and the **retrieved & augmented context** to generate a response.

